Generating Link/02 Compatible Objects

This tutorial will teach you how to create objects for use with Link/02. This tutorial is meant for those that are writing tools that want to leverage the power of Link/02 for creating binary executables. This tutorial does not cover how to use Link/02. I will be using Asm/02 for generating input files for Link/02.

Object files for Link/02 are straight ASCII files consisting of a series of 1-line records that instruct Link/02 on what to do. Each record begins with a single character that specifies what the rest of the line represents. Lets start with a very simple ORG-based single file program. Here is the assembly source for it:

```
          org     02000h
start:    seq
          sep     r4
          dw      delay
          req
          ldi     07fh
          phi     rc
          sep     r4
          dw      delay+3
          lbr     start

          org     02100h
delay:    ldi     0ffh
          phi     rc
          ldi     0ffh
          plo     rc
delaylp:  dec     rc
          glo     rc
          lbnz    delaylp
          ghi     rc
          lbnz    delaylp
          sep     r5

          end     start
```

Which will produce this object file:

```
.big
:2000 7b d4 21 00 7a f8 7f bc d4 21 03 c0 20 00
:2100 f8 ff bc f8 ff ac 2c 8c ca 21 06 9c ca 21 06 d5
@2000
```

This file contains three different record types: '.', ':', and '@'. We will go over what each of these record types represent.

A record that begins with a '.' marker is a directive to the linker and effects how the linker works. In this case, '.big' instructs Link/02 to use big-endian mode when dealing with 16-bit addresses.

A record that begins with a ':' marker is a data line, these are generally going to be the most numerous records in an object file. The number following the ':' character is the offset of where data will be loaded, this is then followed by a series of byte values that will be stored at that offset. For ORG modules (like our current sample) the addresses are absolute addresses and are not modified in any way by the linker.

The last record type in this sample is the '@' record. This record type specifies an execution start address for the program. This address is used when generating executable headers, but otherwise does not affect the linker process.

Now lets look at an example program that is split into two objects. We will start with this part:

```
            org     02000h
            extrn   delay
start:      seq
            sep     r4
            dw      delay
            req
            ldi     07fh
            phi     rc
            sep     r4
            dw      delay+3
            lbr     start
            end     start
```

Since 'delay' is no longer included in our source, we must now inform Asm/02 that delay is an external symbol. Declaring delay as external does two things, first it prevents Asm/02 from declaring delay an unknown symbol, and more importantly it now allows Asm/02 to generate external symbol refernces in the output object, which is actually what we want to see. Here is the object generated from this code:

```
.big
?delay 2002
?delay 2009
:2000 7b d4 00 00 7a f8 7f bc d4 00 03 c0 20 00
@2000
```

The new recored type '?' informs Link/02 of an unresolved reference. The name following the '?' is the symbol name of the reference and this is then followed by the offset of where the reference is located.

In our sample program, we made two calls to 'delay', this is why there are now two unresolved references in our output object. If we look at the address refrenced in the first one, $2002, in the data for this object we see at $2002 are the two bytes: 00 00. Since Asm/02 did not know what the value of 'delay' was it just writes an offset as a place holder. Now lets take a quick look at the second reference, at $2009. In the data bytes at $2009 we now see: 00 03. In the sample program this reference to delay was +3, so Asm/02 wrote the 00 03 as the place holder. This is an important point, the place holder where an unresolved reference

goes, is an offset of the unresolved reference. When Link/02 resolves these two references the value in the placeholder is added to the value of the symbol to produce the final address. This allows for the case here where we are making a call 3 bytes into the delay routine.

Now we need the second object, the one containing delay. Here is the assembly source for it:

```
            org     02100h
delay:      ldi     0ffh
            phi     rc
            ldi     0ffh
            plo     rc
delaylp:    dec     rc
            glo     rc
            lbnz    delaylp
            ghi     rc
            lbnz    delaylp
            sep     r5

            public  delay
```

Since 'delay' is being called from objects outside of this assembly unit, we declare 'delay' to be public, which will generate a public symbol definition in the output object. The output object for this assembly produces this:

```
.big
:2100 f8 ff bc f8 ff ac 2c 8c ca 21 06 9c ca 21 06 d5
=delay 2100
```

The new record type here is the '=' record. This record type declares a public symbol. The symbol name follows the '=' character and then the value of this symbol follows.

In its most basic form, Link/02 matches up all the '?' records in the input objects with the corresponding '=' records. So in our sample, now that the value of 'delay' is known ($2100), the unresolved references at $2002 and $2009 can now be resolved. At $2002, Link/02 will replace the 00 00 with 21 00, and at $2009, the 00 03 will be replaced with 21 03. At this point all references are resolved and the final output can be produced.

That covers ORG objects. Link/02 also supports relocatable PROC objects. ORG objects always reside at the absolute addresses contained within the input objects, which is not terribly useful for library routines that need to be moved around depending on other routines included in the project. Link/02 can relocate PROC objects to anywhere in memory.

Lets now convert our 'delay' subroutine from an ORG object to a PROC object. Here is the new assembly source for delay:

```
            proc    delay
            ldi     0ffh
```

```
                phi     rc
                ldi     0ffh
                plo     rc
        delaylp:  dec     rc
                glo     rc
                lbnz    delaylp
                ghi     rc
                lbnz    delaylp
                sep     r5
                endp
```

Lets take a quick look at the changes, first, there is no longer an ORG statement to specify where this code goes, instead it has been replaced with a PROC statement. The 'delay:' label and its public declaration have been removed and PROC command has an argument of 'delay'. PROCs are always named and the name given to them is always a public symbol, which is why we did not need to declare it as public. Other than these small changes, the rest of the code remains as is. Now lets look at the output object for this code:

```
        .big
        {delay
        :0000 f8 ff bc f8 ff ac 2c 8c ca 00 06 9c ca 00 06 d5
        +0009
        +000d
        }
```

We have a couple new record types. First is the '{' record, which delcares the start of a relocatable procedure. Following the '{' character is the name of the PROC and is entered into the public symbol table. You notice that it contains no value, its value is dynamically created by Link/02 and will be set to the memory load address of this module. If this is linked with our first part of our two part program, delay will have the value of $200E. The first part of the program was loaded from $2000 through $200D, so the next available address becomes $200e, which is now where this 'delay' PROC will be loaded and therefore 'delay' takes on the value of $200E.

The record that begins with the '}' marks the end of a relocatable PROC module. At this point Link/02 will revert back to ORG mode until another PROC record is encountered.

Now those '+' records. Take a note of the addresses used by the data (😃 records, PROC modules are always ORG'd at $0000, this is to accomodate location fixups when this module is loaded into memory. Our program has two LBNZ commands in it, LBNZ jumps to absolute memory addresses, which means if this module is moved then the addrsses being referencec by the LBNZ instructions need to be modified, this is what the '+' records accomplish.

The '+' records indicate at what address a location fixup needs to occur at. In the first one, the location that needs adjustment is $2009, the address portion of the first LBNZ command. The '+' marker works very much like the '?' marker, in that it takes a value and adds it to the bytes at the addressed point. So in this case, at $2009 are the bytes 00 06, which is the offset of where the LBNZ is trying to jump to, these two bytes will be added to the value of 'delay', which contains the load address of the this module and then 00

14 will end up being written at $2009, now the destination of the LBNZ will jump to the correct place in memory. The second relocation fixup at $200D will take care of the second LBNZ.

So the way to look at this, the '?' allows for resolving a symbol that is defined outside of a module and placing the correct address into code. The '+' command allows for resolving a symbol INSIDE of a relocatable PROC module and adjusting the address for where the module got loaded.

That covers 16-bit references, but what about when you want only the high or low portion of a 16-bit reference? Well, Link/02 has you covered there as well. Lets take a look at a program that uses 8-bit references to 16-bit addresses:

```
            org     02000h
            extrn   delay
start:      ldi     delay.1
            phi     rd
            ldi     delay.0
            plo     rd
            end     start
```

In this program we are trying to get the high and low values of the external 'delay' symbol. Here is the generated output:

```
.big
/delay 2001
\delay 2004
:2000 f8 00 bd f8 00 ad
@2000
```

Now, remember from the '?' marker for an unresolved reference, Link/02 would replace the two bytes at the specified address with the value of the referenced symbol. In this program only 1 byte needs to be replaced for each reference to 'delay', the high byte in the first reference, and the low byte in the second. Now notice there are no '?' unresolved reference markers in this output. Instead we have two new markers: '/' and ''. These two markers work exactly like the '?' marker, in that an external symbol is resolved and a value is written to memory. In the case of the '/' marker only the high byte of the referenced symbol value will be stored into the 1 byte address provided. For the '' marker, only the low byte of the referenced symbol value will be written into the 1 byte at the address specified.

And finally we need to look at 8-bit references insode of PROC modules. Lets take this program:

```
            proc    setrf
            ldi     data.1
            phi     rf
            ldi     data.0
            plo     rf
            sep     r5
```

```
data:   db      0
        endp
```

In this program, since setrf is a relcatable PROC object, the location of 'data' is not at a fixed address until the module is linked. The '+' marker allowed a 16-bit reference to be adjusted based upon the load address of a module, but LDI uses only an 8-bit argument so the '+' marker would not work here. Link/02 has two more markers for dealing with 8-bit references to 16-bit addresses that are in relocatable modules. Here is the output from this program:

```
.big
{setrf
:0000 f8 00 bf f8 07 af d5 00
^0001
v0004
}
```

Just like the '/' and '' markers replaced the '?' marker when referring to only a portion of a 16-bit address, in PROC modules, the '^' and 'v' markers replace the '+' marker. The '^' marker will add the high byte of the module address to the byte at the specified address and the 'v' marker will add the low byte of the module address to the byte at the specified address.

That covers the record types you need to generate ordinary objects for Link/02. There is one more topic worth knowing about, since it can change what your assembler should emit for one specific kind of instruction: short-branch relaxation.

The Series/02 processors have both a 2-byte "short branch" form (target must be on the same page as the byte after the instruction) and a 3-byte "long branch" form (target can be anywhere). Most assemblers, including Asm/02, let a programmer write a branch mnemonic without knowing in advance whether its target will end up close enough to use the short form -- so they default to always emitting the long form for anything that might branch outside the current page, and rely on the programmer to hand-optimize specific branches to their short form only when they already know it's safe.

Link/02 can do that optimization itself, automatically, for any long branch whose operand your assembler tags as such. This is the '#' and '!' record types. They work exactly like '+' and '?' -- '#' is a local (same-procedure) 16-bit reference, '!' is an external one -- with one difference: Link/02 is now allowed to notice that a '#' branch's target ends up on the same page as the branch itself, and rewrite the instruction to its 2-byte short form instead of resolving it as a 3-byte long branch. This only ever happens when the '-r' command line switch is given; without it, '#' and '!' resolve exactly like '+' and '?' always did, so tagging your long branches this way costs nothing on an ordinary link.

'!' branches are never shortened, even with '-r'. An external target's real address isn't known until its defining module is loaded and its '=' record processed, which can happen well after the branching procedure's own layout has already been finalized -- so there's no safe point at which an external branch's range could still be checked. Only same-procedure ('#') branches are ever candidates.

If your assembler already distinguishes branch instructions from other 16-bit references (it has to, in order to pick the opcode), the change needed to support relaxation is small: wherever it would emit a '+' for a

long branch's local target, emit '#' instead; wherever it would emit a '?' for a long branch's external target, emit '!' instead. Every other 16-bit reference -- data words, non-branch address loads, and so on -- keeps using '+' and '?' as before, since only real branch instructions can be shortened this way.

Let's revisit the 'delay' PROC example from earlier. Asm/02 already recognizes LBNZ as a long branch and tags its two operands accordingly, so reassembling that exact same source produces this object, with '#' in place of the '+' shown earlier:

```
.big
{delay
:0000 f8 ff bc f8 ff ac 2c 8c ca 00 06 9c ca 00 06 d5
#0009
#000d
}
```

Linked normally (no '-r'), this resolves identically to the original '+' version -- same bytes, same size. But linked with '-r', both LBNZ instructions turn out to target 'delaylp', which ends up on the same page as both branches (the whole procedure is only 14 bytes), so both get shortened. Link/02 prints a short summary of what it did:

```
Relaxation: 2 of 2 local long branches shortened (1 rounds)
```

and the procedure's own final bytes shrink from 16 to 14:

```
f8 ff bc f8 ff ac 2c 8c 3a 06 9c 3a 06 d5
```

If you're curious what Link/02 does internally to get there: with '-r', it doesn't patch branch bytes in place. Instead it makes repeated passes, and on each one it regenerates every procedure's object text completely from scratch for a given "these must stay long" set (empty on the first pass) -- non-excluded '#' branches get shortened and every offset in the procedure (byte positions, fixup patch offsets, local fixups' own stored targets) is renumbered against the procedure's new, smaller size before anything is resolved. That regenerated text is fed through the same ordinary object-loading and linking process used for any other input. If a shortened branch then turns out not to actually fit (some other branch elsewhere in the same procedure grew rather than shrank, moving the target off the page after all), that one branch is added to the exclusion set and every procedure is regenerated fresh again -- not patched -- and the whole thing repeats until a pass produces no such failures. You will not see any of this in the object file itself; it is purely a Link/02-side optimization, invisible to your assembler except for the '#'/'!' tags it emits up front.

Regenerating a procedure needs the whole target of every local reference, to find where it moved to. A '^' record has it: the high byte in the data and the low byte as its parameter. A 'v' record has only the low byte, so Link/02 places it in one of two ways. If a '^' record of the same target is right next to it in the list -- the same low byte, as "ldi label.1 ... ldi label.0" gives -- it takes the target from that. Otherwise it tries the address with that low byte on each page of the procedure, and if they all end up with the same low byte after the shrinks, which is always so when nothing before them shrinks or the procedure is one page, that is

the answer. If they do not, the 'v' can not be placed, so no branch of that procedure is shrunk, and Link/02 says so:

```
Relaxation: the branches of proc am in ambiguous.prg are left
long, as the low byte of a local address at 0164 could not be
placed
```

An assembler can avoid that by writing the high byte too, or by loading the address of a label on a page of its own, such as variables that are on a page boundary, before any shrinkable branch.

If your procedure genuinely needs a short branch at a specific, known-safe spot -- a tight loop where every byte or cycle counts, say -- your assembler can still emit a plain '<' record directly, exactly as it always could. Unlike '#', a '<' record carries no separate target field: the low byte already stored in the ':' data at that offset IS the target, and Link/02 only adds the procedure's own load-address offset to it and checks the result still lands on the same page. Here's an assembled BNZ (short branch) instruction and the object it produces:

```
.big
{loopy
:0000 f8 10 ac 2c 8c 3a 03 d5
<0006
}
```

A procedure that mixes hand-written short branches with '-r' relaxation works fine -- Link/02 remaps a plain '<' record's stored target right along with everything else as procedures shrink, the same as it does for every other record type. The one thing worth knowing: if you want a short branch's own target to be GUARANTEED to stay on the same page no matter what relaxation does elsewhere in the procedure, align the whole procedure to a page boundary with '.align page' immediately before its '{name' record (see "object file structure" for the other alignment sizes). "Immediately before" matters -- alignment appearing anywhere else in the middle of a procedure does not reserve space your assembler's own byte-offset bookkeeping already assumed wasn't there, and produces wrong addresses.

That covers all of the record types currently recognized by Link/02.